ENSI CAEN
ÉCOLE PUBLIQUE D'INGÉNIEURS
CENTRE DE RECHERCHE

Challenge de programmation

GRAND PRIX F1

Année universitaire 2025 — 2026

RAPPORT - ParadoxTitan



Khalil ANBARI

khalil.anbari@ecole.ensicaen.fr

Aya KHOUYA

aya.khouya@ecole.ensicaen.fr

Enseignants référents : Tristan Benoit & Éric Ziad-Forest

Table des matières

1 Introduction **3**
1.1 Contexte . . . 3
1.2 Objectifs . . . 3

2 Architecture logicielle **3**
2.1 Modules . . . 3
2.2 Compilation . . . 3

3 Modélisation : graphe d'états **5**
3.1 État d'un pilote . . . 5
3.2 Indexation des états . . . 5
3.3 Construction par BFS . . . 6

4 Recherche du chemin optimal **7**
4.1 Algorithme A* . . . 7
4.2 File de priorité : tas de Fibonacci . . . 7
4.3 Reconstruction du chemin . . . 8

5 Validation géométrique des déplacements **9**

6 Optimisations dynamiques **10**
6.1 Coefficients adaptatifs selon le carburant . . . 10
6.2 Évitement actif des adversaires . . . 10
6.3 Recalcul conditionnel . . . 10

7 Traces d'exécution **12**
7.1 Tests seul (sans adversaires) . . . 12
7.2 Tests avec trois instances de ParadoxTitan . . . 12
7.3 Test sur carte complexe (F-Zero) . . . 12

8 Bilan technique **14**
8.1 Difficultés rencontrées . . . 14
8.2 Perspectives d'amélioration . . . 14

9 Conclusion **15**

1. Introduction

1.1 Contexte

Le challenge consiste à implémenter un pilote autonome de Formule 1 en langage C. Le pilote reçoit à chaque tour, via `stdin`, la carte du circuit et les positions des trois voitures, et doit répondre en moins d'une seconde par un vecteur d'accélération (ax, ay) sur `stdout`. L'objectif est d'atteindre la ligne d'arrivée en un minimum de tours, tout en respectant les contraintes de vitesse, de carburant et d'évitement des obstacles.

1.2 Objectifs

- Trouver le chemin optimal vers l'arrivée sur tout type de carte.
- Gérer la consommation de carburant pour ne pas tomber en panne avant l'arrivée.
- Éviter les collisions avec les deux autres pilotes.
- Répondre en moins d'une seconde à chaque tour, quelle que soit la taille de la carte.

2. Architecture logicielle

2.1 Modules

Le pilote est structuré en huit modules. Chaque module a une responsabilité unique.

Module	Rôle
track.c	Lecture de la carte depuis `stdin`, accès aux types de cases.
Graph.c	Construction du graphe d'états par BFS, indexation des états.
Astar.c	Algorithme A* : exploration, relaxation, reconstruction du chemin.
fibonacciHeap.c	File de priorité pour A* : insertion O(1) amorti.
mouvement.c	Validation des déplacements par l'algorithme de Bresenham.
utils.c	Calcul de consommation, dialogue GDC, gestion des adversaires.
Stack.c / queue.c / list.c	Structures de données génériques.
ParadoxTitan.c	Boucle principale : lecture des positions, déclenchement du recalcul.

Table 1 – *Modules du pilote et leur rôle.*

2.2 Compilation

Le code est compilé en C ANSI avec les drapeaux `-ansi -pedantic -Wall -Wextra`, sans aucun warning. Le `Makefile` gère la compilation séparée des modules et l'installation de l'exécutable dans `../drivers/`.

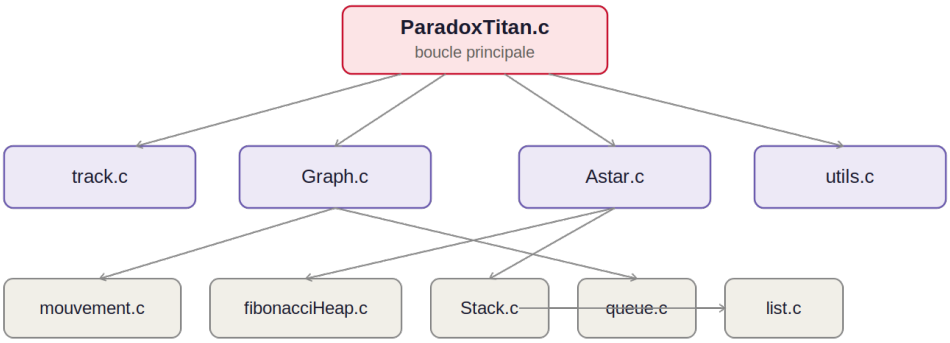


Figure 1 – *Dépendances entre les modules.*

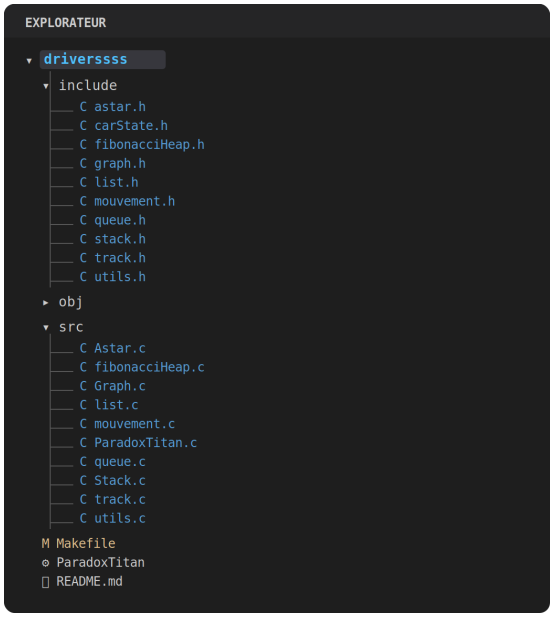


Figure 2 – *Arborescence complète du projet.*

3. Modélisation : graphe d'états

3.1 État d'un pilote

Modéliser le problème avec uniquement la position (x, y) est insuffisant. Deux voitures à la même case mais avec des vitesses différentes n'ont pas les mêmes mouvements accessibles au prochain tour : l'inertie contraint les options.

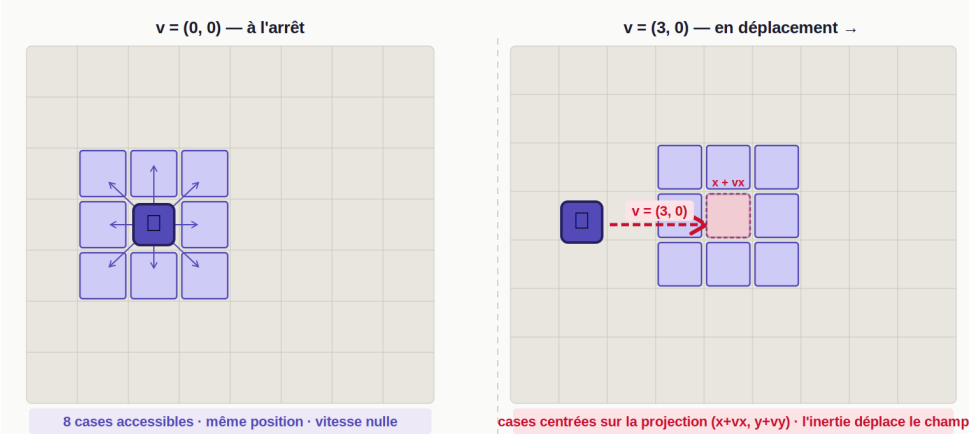


Figure 3 – *Même case, deux vitesses différentes → futurs différents.*

Un état est donc défini par cinq composantes :

$$\text{état} = (x,\ y,\ v_x,\ v_y,\ b)$$

Avec (x, y) la position, (v_x, v_y) le vecteur vitesse courant, et b l'indicateur de disponibilité du boost. Chaque sommet du graphe correspond à un état complet, pas à une simple case.

3.2 Indexation des états

Chaque état est associé à un entier unique par la fonction d'indexation $\mathcal{I}$, construite par encodage en base mixte. Cette bijection garantit l'absence de collision : deux états distincts produisent toujours deux index distincts.

Fonction d'indexation des états

$$\mathcal{I}(x,\, y,\, v_x,\, v_y,\, b) \;=\; \underbrace{\big[(y \cdot W + x) \cdot V + (v_x + v_{\max})\big] \cdot V + (v_y + v_{\max})}_{\text{encodage de la position et de la vitesse}} \cdot (B_{\max}{+}1){+}b$$

W : largeur de la carte (nombre de colonnes)

H : hauteur de la carte (nombre de lignes)

V : $= 2\,v_{\max} + 1 = 11$ (plage : $v_x,\, v_y \in [-5,\, +5]$)

$v_{\max}$: $= 5$ vitesse maximale imposée par le règlement

$B_{\max}$: $= 5$ boosts disponibles en début de course

b : $\in \{0, 1, 2, 3, 4, 5\}$ boosts restants

Taille de l'espace : $W \times H \times V^2 \times (B_{\max}{+}1)$. Pour une carte $80{\times}60$: $80{\times}60{\times}121{\times}6 \approx 3{,}5$ millions d'états.

Seuls les états réellement atteignables depuis la position courante sont explorés par le BFS, ce qui réduit fortement la taille effective du graphe en pratique.

3.3 Construction par BFS

Le graphe est construit à chaque planification par un BFS depuis l'état courant. Pour chaque état exploré, les neuf accélérations élémentaires ($a_x, a_y \in \{-1, 0, 1\}$) sont testées. Une transition est ajoutée au graphe si :

1. la norme de la nouvelle vitesse est ≤ 5 (≤ 1 dans le sable) ;
2. la case de destination est dans la grille et n'est pas de type `HORS_PISTE` ;
3. le déplacement ne traverse aucune case hors-piste (vérifié par Bresenham, section 5).

Le coût d'une transition est égal à la consommation de carburant calculée selon la formule du règlement.

4. Recherche du chemin optimal

4.1 Algorithme A*

A* est utilisé pour trouver le chemin de coût minimum dans le graphe d'états. Contrairement à Dijkstra, A* guide l'exploration vers la cible grâce à une heuristique, ce qui réduit le nombre d'états visités et respecte la contrainte de temps d'une seconde par tour.

À chaque étape, A* extrait l'état minimisant $f(n) = g(n) + h(n)$:

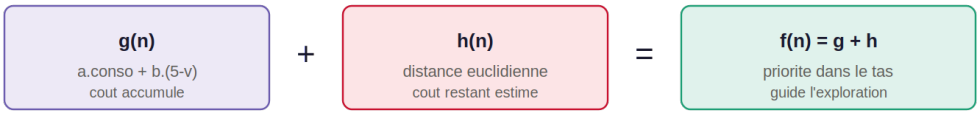


Figure 4 – *Décomposition de* $f(n) = g(n) + h(n)$.

$g(\textbf{arc}) = a \cdot \text{consommation} + b \cdot (5 - \text{vitesse})$. Le premier terme minimise la consommation de carburant. Le second récompense la vitesse : une vitesse élevée réduit le coût de l'arc. Les coefficients a et b permettent de régler le compromis vitesse/carburant (voir section 6.1).

$h(n) = \sqrt{(x_n - x_{\text{goal}})^2 + (y_n - y_{\text{goal}})^2}$: distance euclidienne vers la case d'arrivée la plus proche. L'heuristique est admissible (elle ne surestime pas le coût réel), donc A* retourne un chemin optimal.

4.2 File de priorité : tas de Fibonacci

A* effectue un grand nombre d'insertions (une par arc relaxé). Un tas de Fibonacci offre une insertion en $O(1)$ amorti, contre $O(\log n)$ pour un tas binaire, ce qui réduit le temps de calcul sur les grands graphes.

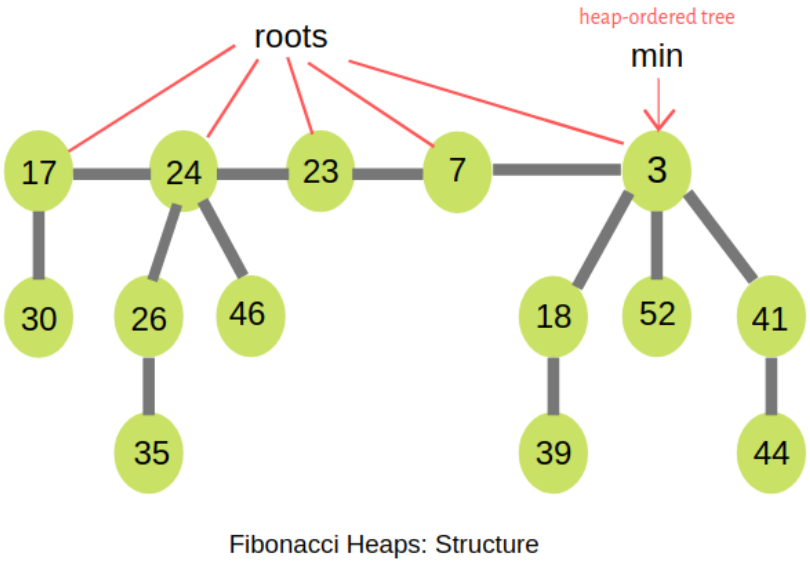


Figure 5 – *Structure d'un tas de Fibonacci : liste de racines reliée horizontalement, arbres ordonnés par clé, pointeur* ***min*** *vers la racine de plus petite priorité. Les insertions se font en* $O(1)$ *amorti, l'extraction du minimum en* $O(\log n)$ *amorti.*

Opération	Tas binaire	Tas de Fibonacci
Insertion	$O(\log n)$	$O(1)$ amorti
Extraction du minimum	$O(\log n)$	$O(\log n)$ amorti
Diminution de clé	$O(\log n)$	$O(1)$ amorti

Table 2 – *Complexité comparée des opérations de file de priorité.*

4.3 Reconstruction du chemin

Quand A* atteint une case de type `FINISH`, le chemin est reconstruit en remontant le tableau `parents[]` depuis l'état final jusqu'à l'état initial. Les états sont empilés dans une `Stack`. À chaque tour, le pilote dépile le prochain état et calcule l'accélération à envoyer au GDC.

5. Validation géométrique des déplacements

Le règlement interdit de traverser les cases hors-piste lors d'un déplacement. Un pilote ne se téléporte pas : il parcourt réellement toutes les cases entre sa position et sa destination.

La fonction `isMovementValid()` applique l'algorithme de Bresenham pour énumérer, dans l'ordre, les cases discrètes traversées par le segment $[A \to B]$. Dès qu'une case de type `HORS_PISTE` est rencontrée, la fonction retourne immédiatement 0 et le déplacement est rejeté. Tant que toutes les cases sont praticables, la fonction retourne 1.

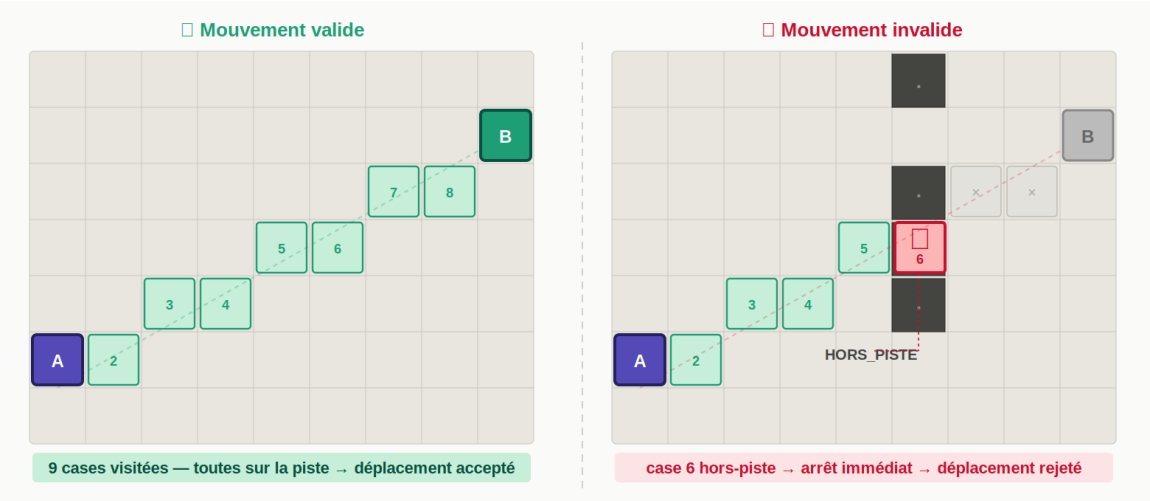


Figure 6 – *Algorithme de Bresenham appliqué à la validation d'un déplacement.* ***Gauche*** *: les 9 cases visitées sont toutes sur la piste, le déplacement est accepté.* ***Droite*** *: la case 6 est hors-piste, le parcours s'arrête et le déplacement est rejeté. Les cases grises (×) ne sont jamais visitées.*

Cette vérification est appelée pour chaque arc lors de la construction du graphe. Seuls les arcs géométriquement valides sont ajoutés.

6. Optimisations dynamiques

6.1 Coefficients adaptatifs selon le carburant

Les coefficients a et b de la fonction $g(\text{arc})$ sont ajustés en fonction du pourcentage de carburant restant. Quand le carburant est abondant, le pilote privilégie la vitesse. Quand il diminue, il réduit la consommation.

Carburant restant	a	b	Stratégie
$> 70\%$	2.0	2.5	Vitesse maximale
$50 - 70\%$	2.5	2.5	Équilibre vitesse/consommation
$40 - 50\%$	3.0	0.5	Économie de carburant
$< 40\%$	3.5	0.3	Survie : minimiser la consommation

Table 3 – *Coefficients a et b selon le niveau de carburant.*

6.2 Évitement actif des adversaires

Avant chaque calcul A*, la vitesse de chaque adversaire est estimée par différence de positions entre le tour courant et le tour précédent. Les cases actuelle et probable de chaque adversaire sont temporairement marquées `HORS_PISTE` dans la carte. A* planifie donc une trajectoire qui les évite. Les cases sont restaurées après le calcul.

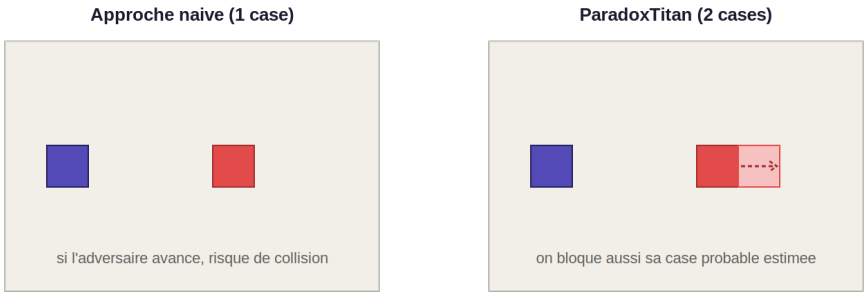


Figure 7 – *Gauche : blocage naïf (case actuelle uniquement). Droite : blocage anticipé (case actuelle + case probable).*

6.3 Recalcul conditionnel

A* n'est relancé que si le plan courant devient invalide. Le pilote suit son plan existant (dépilage) dans tous les autres cas, ce qui garantit le respect de la limite de temps.

- **pathInvalid** : la position réelle diffère de la position prévue (crash ou arrêt par le GDC).
- **nextCellBlocked** : la prochaine case du plan est occupée par un adversaire.
- **carburantInsuffisant** : le carburant restant est insuffisant pour continuer le plan.

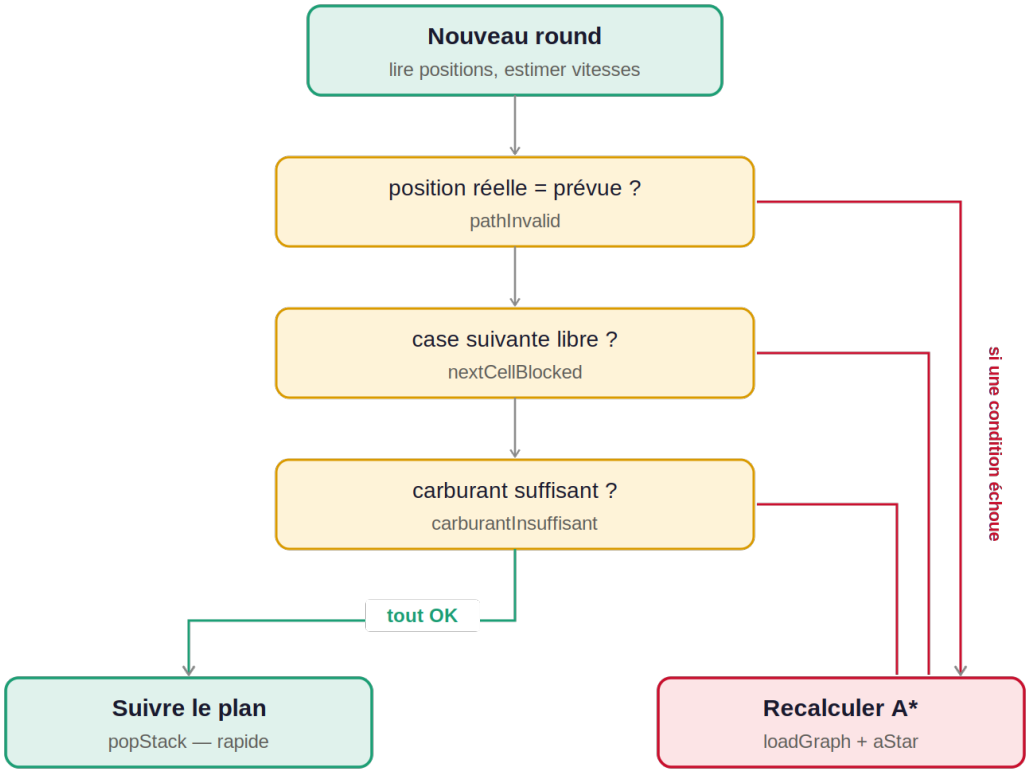


Figure 8 – *Conditions déclenchant un recalcul A*.*

7. Traces d'exécution

7.1 Tests seul (sans adversaires)

Les deux premières séries de tests ont été réalisées avec ParadoxTitan seul, pour valider la planification de trajectoire indépendamment de la gestion des adversaires.

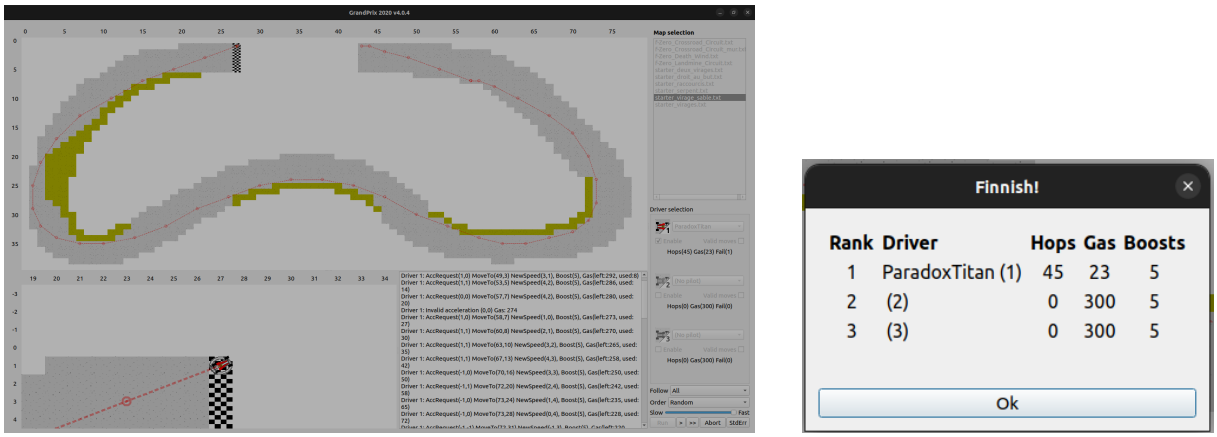


Figure 9 – *`starter_virage_sable` — trajectoire et résultat : 1er en 45 tours, 23 unités de carburant restantes.*

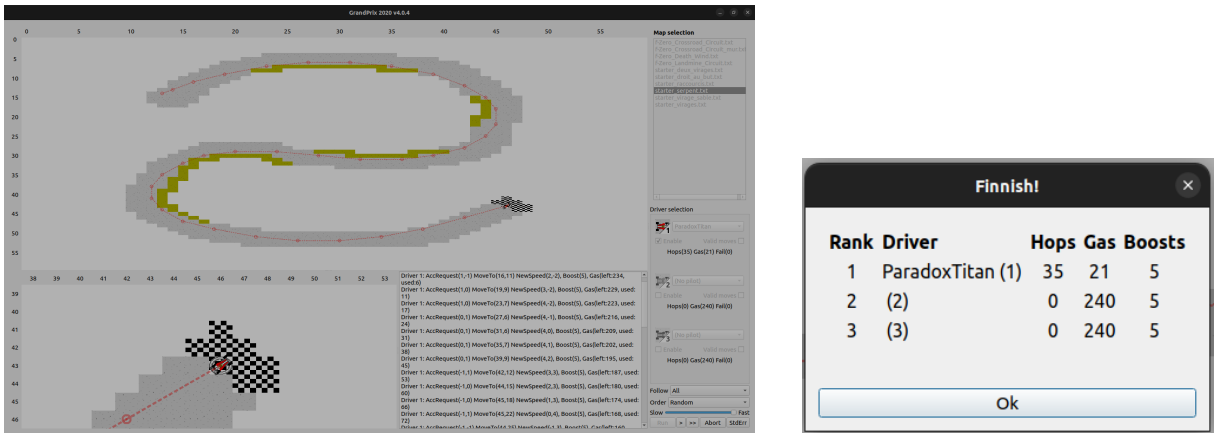


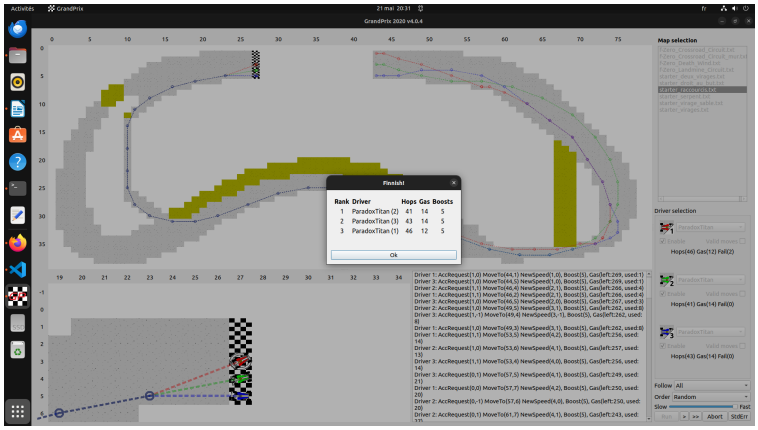
Figure 10 – *`starter_serpent` — trajectoire et résultat : 1er en 35 tours, 21 unités de carburant restantes.*

7.2 Tests avec trois instances de ParadoxTitan

Les tests suivants lancent trois instances simultanées du pilote. Ils valident la gestion des collisions et l'évitement actif des adversaires.

7.3 Test sur carte complexe (F-Zero)

Un test sur la carte `f-Zero_Crossroad_Circuit_mur` valide le comportement du pilote sur une carte inconnue de grande taille avec un tracé difficile.



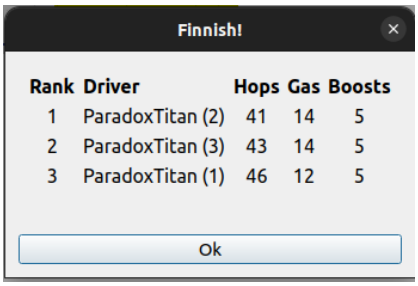
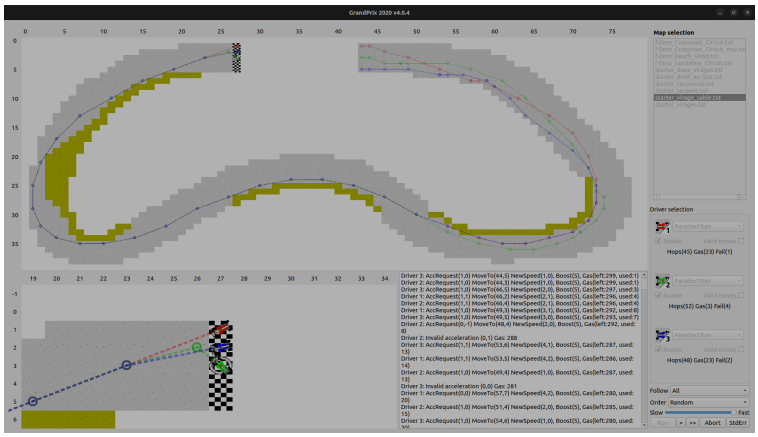


Figure 11 – `starter_raccourcis` — *3× ParadoxTitan. 1er en 41 tours (14 gas), 2e en 43 tours (14 gas), 3e en 46 tours (12 gas).*



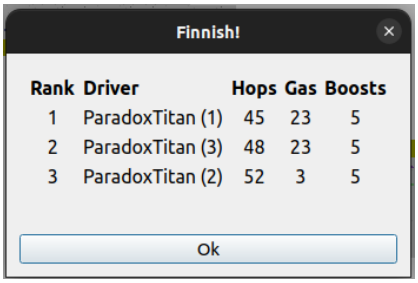
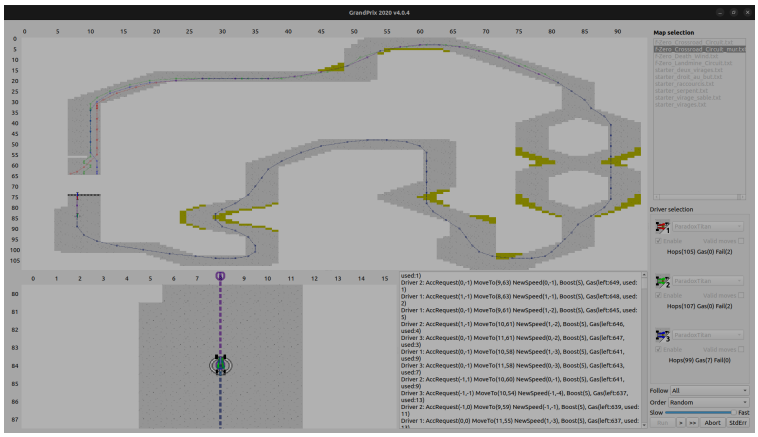


Figure 12 – `starter_virage_sable` — *3× ParadoxTitan. 1er en 45 tours (23 gas), 2e en 48 tours (23 gas), 3e en 52 tours (3 gas).*



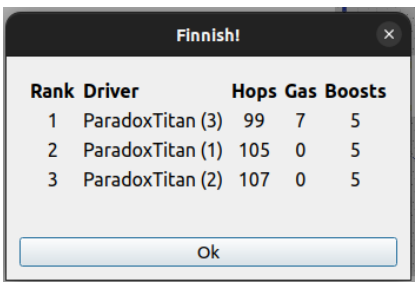


Figure 13 – `f-Zero_Crossroad_Circuit_mur` — *3× ParadoxTitan. 1er en 99 tours (7 gas), 2e en 105 tours (0 gas), 3e en 107 tours (0 gas).*

8. Bilan technique

8.1 Difficultés rencontrées

— Recalcul A* à chaque tour dans la première version : dépassement de la limite de temps sur les grandes cartes. Résolu par la logique de recalcul conditionnel.
— Calibrage des coefficients a et b : trouver les valeurs qui évitent à la fois la panne sèche et la perte de vitesse a nécessité de nombreux tests.
— Débogage difficile : le GDC lance le pilote, `gdb` n'est pas utilisable directement. Les traces sur `stderr` et l'option `-fsanitize=address` ont été utilisées pour détecter les erreurs mémoire.
— Gestion du rayon de blocage des adversaires : un rayon trop large bloquait la propre case de départ du pilote. Le rayon a été limité à deux cases (case actuelle + case probable).

8.2 Perspectives d'amélioration

— Calibrage automatique des coefficients a et b par apprentissage par renforcement : l'agent s'ajusterait en cours de course selon les résultats observés, sans nécessiter une définition manuelle des seuils.
— Anticipation des adversaires sur plusieurs tours au lieu d'un seul, pour éviter les situations où deux recalculs successifs sont nécessaires.

9. Conclusion

Ce projet Grand Prix F1 nous a permis de concevoir un pilote autonome en abordant le problème comme un véritable système de décision temps réel. La modélisation par graphe d'états, combinée à l'algorithme A* et à la validation des trajectoires par Bresenham, a permis de construire une solution cohérente, fiable et adaptée aux contraintes imposées par le gestionnaire de course.

Le développement de ParadoxTitan a également mis en évidence l'importance du compromis entre performance, robustesse et coût de calcul. Les optimisations intégrées, notamment l'adaptation au carburant, l'évitement des adversaires et le recalcul conditionnel du chemin, ont contribué à rendre le pilote plus stable face à des situations de course variées.

Enfin, cette solution constitue une base évolutive qui pourrait être améliorée par un calibrage automatique des paramètres de coût ou par une anticipation plus avancée du comportement des autres pilotes.